

Paper Validation Report

for DiMag



Correspondence percentages: 47%

Number of experiments found: 18

Experiment 1. Formulation stated: "we manually collected 1282 raw samples from 126 unique scientific articles with descriptions of synthesis, materials characterizations, and measurements of magnetic hyperthermia and MRI performances in vitro." **Implementation found:** ['NOT FOUND']

Missing components: ['Manual data collection pipeline for 1282 samples from 126 scientific articles', 'Text/PDF parsing and extraction of synthesis and materials characterization descriptions', 'Extraction and structuring of magnetic hyperthermia and MRI in vitro performance measurements', 'Data curation and validation workflow for raw literature data'] **Correspondence:** 5.0%

Experiment 2. Formulation stated: "Average magnetic moment of metals in the core was calculated using magnetic moments of each element with help of stoichiometry to better describe composition of nanoparticles" **Implementation found:** ['SAR_modeling.py:lines 1-15', 'input_script.py:lines 1-15', 'r2_model_analysis.py:lines 1-15', 'r1_relaxivity_prediction.py:lines 1-10', 'r2_prediction_script.py:lines 1-10', 'SAR_prediction.py:lines 1-12', 'input_code.py:lines 1-12']

Missing components: ['Calculation of average magnetic moment of metals in the core using stoichiometry', 'Logic for combining individual element magnetic moments based on nanoparticle composition', 'Feature engineering step for 'average magnetic moment' descriptor']

Correspondence: 40.0%

Experiment 3. Formulation stated: "Average spin of metal elements on the surface was obtained to describe interactions between the surface elements and proton spins responsible for relaxation properties of the nanoparticle" **Implementation found:** ['SAR_modeling.py:lines 1-25', 'input_script.py:lines 1-20 (r1 relaxivity prediction)', 'r2_model_analysis.py:lines 1-25', 'r1_relaxivity_prediction.py:lines 1-15', 'r2_prediction_script.py:lines 1-15', 'SAR_prediction.py:lines 1-20', 'input_code.py:lines 1-20 (r1 and r2 relaxivity versions)']

Missing components: ['Calculation or extraction logic for 'Average spin of metal elements on the surface"', 'Explicit modeling of interactions between surface elements and proton spins', 'Physics-based descriptors for surface-to-proton relaxation mechanisms', 'Feature engineering code that specifically derives 'average spin' from raw nanoparticle data"] **Correspondence:** 45.0%

Experiment 4. Formulation stated: "we employed a partition coefficient LogP and number of H acceptors, which were calculated using SMILES representations of organic molecules."

Implementation found: ['SAR_modeling.py:lines 1-25', 'input_script.py:lines 1-25', 'r2_model_analysis.py:lines 1-25', 'r1_relaxivity_prediction.py:lines 1-20', 'r2_prediction_script.py:lines 1-20', 'SAR_prediction.py:lines 1-20', 'input_code.py:lines 1-25']

Missing components: ['Calculation of LogP from SMILES representations', 'Calculation of Number of H acceptors from SMILES representations', 'SMILES string parsing logic or integration with RDKit/Mordred for descriptor generation'] **Correspondence:** 40.0%

Experiment 5. Formulation stated: "Volume and surface area were calculated using the shape and size of the nanoparticle to describe morphological characteristics of nanomagnets."

Implementation found: ['SAR_modeling.py:lines 1-15', 'input_script.py:lines 1-12', 'r2_model_analysis.py:lines 1-15', 'r1_relaxivity_prediction.py:lines 1-10', 'r2_prediction_script.py:lines 1-10', 'SAR_prediction.py:lines 1-12', 'input_code.py:lines 1-12']

Missing components: ['Explicit calculation logic for Volume and Surface Area from nanoparticle shape/size (code assumes descriptors are already present in CSV)', 'Mathematical formulas for morphological characterization of nanomagnets', 'Geometric feature engineering functions']

Correspondence: 40.0%

Experiment 6. Formulation stated: "we calculated and analyzed the Pearson correlation matrix"

Implementation found: ['SAR_modeling.py:lines 1-20', 'input_script.py:lines 1-20', 'r2_model_analysis.py:lines 1-20', 'r1_relaxivity_prediction.py:lines 1-15', 'r2_prediction_script.py:lines 1-15', 'SAR_prediction.py:lines 1-15', 'input_code.py:lines 1-15']

Missing components: ['Pearson correlation matrix calculation logic (NOT FOUND in any script description)', 'Feature-to-feature correlation analysis implementation', 'Heatmap or tabular output of

correlation coefficients'] **Correspondence:** 10.0%

Experiment 7. Formulation stated: "we observed, for instance, a group of geometrical features with high cross-correlations and reduced them to a single area-volume feature." **Implementation found:** ['SAR_modeling.py:lines 1-25', 'input_script.py:lines 1-20', 'r2_model_analysis.py:lines 1-25', 'r1_relaxivity_prediction.py:lines 1-15', 'r2_prediction_script.py:lines 1-15', 'SAR_prediction.py:lines 1-20', 'input_code.py:lines 1-20'] **Missing components:** ['Explicit cross-correlation analysis logic to identify redundant geometrical features', "Feature reduction logic/function to merge multiple geometrical features into a single 'area-volume' feature", "Hardcoded or dynamic selection of the specific 'area-volume' feature mentioned in the description"] **Correspondence:** 40.0%

Experiment 8. Formulation stated: "we chose kNN imputation technique, because it considers the relationships between the different features, it is robust to outliers, and it can handle both categorical and continuous variables." **Implementation found:** ['SAR_modeling.py:lines 1-15', 'input_script.py:lines 1-15', 'r2_model_analysis.py:lines 1-15', 'r1_relaxivity_prediction.py:lines 1-10', 'r2_prediction_script.py:lines 1-10', 'SAR_prediction.py:lines 1-12', 'input_code.py:lines 1-15'] **Missing components:** ['kNN imputation technique (The code descriptions mention normalization and log-transformation, but no explicit kNN imputation logic is found in any script)', 'Handling of categorical variables (The scripts focus on physical/chemical descriptors and continuous feature normalization via MinMaxScaler, but do not explicitly implement categorical encoding or kNN-based categorical handling)'] **Correspondence:** 40.0%

Experiment 9. Formulation stated: "we first removed the total of 211 samples such that each sample in the remaining dataset had no more than 15% of missing values. After that, we applied the kNN imputation algorithm to fill the gaps and obtain the complete dataset." **Implementation found:** ['SAR_modeling.py: (kNN imputation logic not explicitly detailed in description)', 'input_script.py: (kNN imputation logic not explicitly detailed in description)', 'r2_model_analysis.py: (kNN imputation logic not explicitly detailed in description)', 'r1_relaxivity_prediction.py: (kNN imputation logic not explicitly detailed in description)', 'r2_prediction_script.py: (kNN imputation logic not explicitly detailed in description)', 'SAR_prediction.py: (kNN imputation logic not explicitly detailed in description)', 'input_code.py: (kNN imputation logic not explicitly detailed in description)'] **Missing components:** ['Initial removal of 211 samples (NOT FOUND)', 'Filtering logic for samples with >15% missing values (NOT FOUND)', 'Implementation of kNN imputation algorithm (NOT FOUND)'] **Correspondence:** 0.0%

Experiment 10. Formulation stated: "we transformed highly correlated geometrical features into area to volume and max to min length ratios reflecting size and shape anisotropy of nanoparticles." **Implementation found:** ['SAR_modeling.py:lines 1-25', 'input_script.py:lines 1-20', 'r2_model_analysis.py:lines 1-22', 'r1_relaxivity_prediction.py:lines 1-15', 'r2_prediction_script.py:lines 1-15', 'SAR_prediction.py:lines 1-18', 'input_code.py:lines 1-20'] **Missing components:** ["Explicit feature engineering logic for 'area to volume' ratio transformation", "Explicit feature engineering logic for 'max to min length' ratio transformation", "Logic for handling/reducing 'highly correlated geometrical features' as described in the prompt"] **Correspondence:** 40.0%

Experiment 11. Formulation stated: "we removed samples with outliers by setting empirical thresholds on feature values while analyzing their distributions." **Implementation found:** ['SAR_modeling.py:lines 1-15 (data loading and distribution analysis)', 'input_script.py:lines 1-10 (data loading and feature normalization)', 'r2_model_analysis.py:lines 1-12 (feature selection and normalization)', 'r1_relaxivity_prediction.py:lines 1-8 (data loading and normalization)', 'r2_prediction_script.py:lines 1-8 (data loading and normalization)', 'SAR_prediction.py:lines 1-10 (feature selection and normalization)', 'input_code.py:lines 1-10 (preprocessing using MinMaxScaler)'] **Missing components:** ["Explicit outlier removal logic (the code descriptions mention loading and normalization, but no specific logic for 'removing samples' based on 'empirical thresholds' is described in the metadata)", "Empirical thresholding implementation for feature values", "Conditional filtering of the dataset based on distribution analysis results"] **Correspondence:** 30.0%

Experiment 12. Formulation stated: "we divided it into three, one for each regression problem: prediction of SAR value, r1 or r2 relaxivity containing 460, 355, and 465 samples, respectively" **Implementation found:** ['SAR_modeling.py: (Full script logic for SAR prediction)', 'SAR_prediction.py: (Full script logic for SAR prediction)', 'input_script.py: (Full script logic for SAR prediction)', 'input_script.py: (Full script logic for r1 relaxivity prediction)', 'r1_relaxivity_prediction.py: (Full script logic for r1 relaxivity prediction)', 'input_code.py: (Full script logic for r1 relaxivity prediction)', 'r2_model_analysis.py: (Full script logic for r2 relaxivity prediction)', 'r2_prediction_script.py: (Full script logic for r2 relaxivity prediction)', 'input_code.py: (Full script

logic for r2 relaxivity prediction)'] **Missing components:** ['Explicit verification of sample counts (460 for SAR, 355 for r1, 465 for r2) within the scripts', 'Unified script or master orchestration file that divides the dataset into these three specific subsets as described'] **Correspondence:** 90.0%

Experiment 13. Formulation stated: "We normalized and log-scaled the data for each problem before splitting to train and test sets in the 80/20 ratio." **Implementation found:**

['SAR_modeling.py: lines (normalization, log-scaling, 80/20 split, 10-fold CV)', 'input_script.py: lines (normalization, log-scaling, 80/20 split, 10-fold CV)', 'r2_model_analysis.py: lines (normalization, log-scaling, 80/20 split, 10-fold CV)', 'r1_relaxivity_prediction.py: lines (normalization, log-scaling, 80/20 split, 10-fold CV)', 'r2_prediction_script.py: lines (normalization, log-scaling, 80/20 split, 10-fold CV)', 'SAR_prediction.py: lines (normalization, log-scaling, 80/20 split, 10-fold CV)', 'input_code.py: lines (normalization, log-scaling, 80/20 split, 10-fold CV)'] **Missing components:** None **Correspondence:** 100.0%

Experiment 14. Formulation stated: "hyperparameter tuning was done using the model-agnostic Optuna framework" **Implementation found:** ['SAR_modeling.py: (ExtraTrees, RandomForest, LGBM, and XGBoost training)', 'input_script.py: (r1 relaxivity prediction using ExtraTrees, RandomForest, LGBM, and XGBoost)', 'r2_model_analysis.py: (r2 relaxivity prediction using ExtraTrees, RandomForest, LGBM, and XGBoost)', 'r1_relaxivity_prediction.py:

(ExtraTreesRegressor implementation)', 'r2_prediction_script.py: (ExtraTreesRegressor implementation)', 'SAR_prediction.py: (LightGBM Regressor implementation)', 'input_script.py: (LightGBM Regressor for SAR)', 'input_code.py: (ExtraTreesRegressor for r1 relaxivity)', 'input_code.py: (ExtraTreesRegressor for r2 relaxivity)'] **Missing components:** ['Optuna framework integration (No file mentions or imports the Optuna library for hyperparameter tuning)', 'Automated hyperparameter optimization logic (Hyperparameters are described as 'predefined' or 'specific' rather than being tuned via an active search process)'] **Correspondence:** 75.0%

Experiment 15. Formulation stated: "The best model was selected using the following metrics: mean Q2 and RMSE for tenfold cross-validation and R2 and RMSE for test samples."

Implementation found: ['SAR_modeling.py: (10-fold cross-validation, mean Q2, RMSE, R2)', 'input_script.py: (10-fold cross-validation, R2, RMSE)', 'r2_model_analysis.py: (10-fold cross-validation, R2, RMSE)', 'r1_relaxivity_prediction.py: (10-fold cross-validation, R2, RMSE)', 'r2_prediction_script.py: (10-fold cross-validation, R2, RMSE)', 'SAR_prediction.py: (10-fold cross-validation, R2, RMSE)', 'input_code.py: (10-fold cross-validation, R2, RMSE)'] **Missing components:** ['Explicit logic for 'best model selection' based on the specific combination of mean Q2/RMSE (CV) and R2/RMSE (Test) is not explicitly defined as an automated selection function', 'The metric 'mean Q2' is mentioned in the description but standard regression scripts typically report R2; explicit Q2 calculation (predictive squared correlation) needs verification against standard R2 implementation'] **Correspondence:** 85.0%

Experiment 16. Formulation stated: "we plotted SHAP (SHapley Additive exPlanations) values for each selected ML model" **Implementation found:** ['SAR_modeling.py:lines 1-20 (ExtraTrees, RandomForest, LGBM, and XGBoost models)', 'input_script.py:lines 1-20 (ExtraTrees, RandomForest, LGBM, and XGBoost models)', 'r2_model_analysis.py:lines 1-20 (ExtraTrees, RandomForest, LGBM, and XGBoost models)', 'r1_relaxivity_prediction.py:lines 1-15 (ExtraTrees model)', 'r2_prediction_script.py:lines 1-15 (ExtraTrees model)', 'SAR_prediction.py:lines 1-15 (LGBM model)', 'input_code.py:lines 1-15 (ExtraTrees model)'] **Missing components:** ['SHAP (SHapley Additive exPlanations) value calculation logic', 'SHAP summary or dependence plotting functions', 'Import of 'shap' library'] **Correspondence:** 50.0%

Experiment 17. Formulation stated: "we formed another testing set of samples that have not been used for training and optimizing hyperparameters of the ML models." **Implementation found:**

['SAR_modeling.py:lines 1-25 (stratified splitting and 10-fold cross-validation)', 'input_script.py:lines 1-20 (stratified splitting and external validation set evaluation)', 'input_code.py:lines 1-20 (stratified splitting and validation dataset plotting)', 'r1_relaxivity_prediction.py:lines 1-15 (stratified splitting and cross-validation)', 'r2_prediction_script.py:lines 1-15 (stratified splitting and cross-validation)']

Missing components: ['Explicit hyperparameter optimization routine (e.g., GridSearchCV or Optuna) to verify that the testing set was excluded from the optimization process']

Correspondence: 90.0%

Experiment 18. Formulation stated: "we computed percent of predictions that differed from the true values by less than 50%." **Implementation found:** ['SAR_modeling.py:lines 1-25', 'input_script.py:lines 1-20', 'r2_model_analysis.py:lines 1-22', 'r1_relaxivity_prediction.py:lines 1-15', 'r2_prediction_script.py:lines 1-15', 'SAR_prediction.py:lines 1-18', 'input_code.py:lines 1-20']

Missing components: ['Calculation of the specific metric: 'percent of predictions that differed from the true values by less than 50%' (Accuracy within a threshold)', 'Logic to compute the error margin relative to true values ($y_{pred} - y_{true} / y_{true} < 0.5$)'] **Correspondence:** 40.0%